



Département mathématiques et informatique
UFR des Sciences

TP3 “Récursivité et ZF-expressions”

Exercice 1

- Examinons le code Haskell ci-dessous, qui présente deux déclarations de la fonction calculant la factorielle d'un nombre donné "n". La première déclaration, `factNT`, est récursive non-terminale, tandis que la seconde déclaration, `factT`, est terminale et utilise un accumulateur dans la fonction auxiliaire `factAux`.

```
factNT, factT :: Int -> Int

factNT 0 = 1 -- 1er cas de base
factNT n = n * factNT (n-1) -- cas récursif

factT n = factAux n 1

factAux :: Int -> Int -> Int
factAux 0 acc = acc -- cas de base
factAux n acc = factAux (n-1) (acc * n) -- cas récursif

main::IO()
main = do
    -- fact
    let x = (factT ((2::Int)^(26::Int)))
    print x
```

- Recopier ce code dans votre fichier et générer un fichier binaire afin de comparer les performances de chaque définition. Nous allons compiler le code en activant l'optimisation "-O2" avec la commande suivante :

```
Terminal>ghc -O2 -prof -fprof-auto -rtsopts tp3.hs -o tp3
```
- Comparer les performances des deux fonctions `factT` et `factNT` en modifiant l'application dans la fonction `main` avec la commande ci-dessous, et expliquer en commentaires en déroulant les deux définitions sur un exemple avec des nombres petits.

```
Terminal>./tp3 +RTS -sstderr
```
- Considérons le code suivant d'une fonction `puissanceNT` qui prend en paramètres un nombre "a" et un exposant "b" et retourne a^b . Proposer une fonction `puissanceT` avec une définition terminale en utilisant une fonction auxiliaire définie avec le `where`.

```
puissanceNT::Integer -> Integer -> Integer
puissanceNT a 0 = 1
puissanceNT a b
    | even b = puissanceNT (a*a) (b `div` 2)
    | otherwise = a * puissanceNT a (b-1)
```

Exercice 2

- Que fait la fonction suivante, mettez la réponse en commentaire dans votre code `tp4.hs` :

```
mystery :: [Int] -> [Int]

mystery [] = []
mystery (x:xs) = mystery [y | y <- xs, y <= x]
                ++ [x]
                ++ mystery [y | y <- xs, y > x]
```

Reprenez le code de `mystery` dans votre fichier `tp3.hs`, et tester la fonction dans `ghci`, vérifiez vos intuitions.

Exercice 3

Définir les fonctions suivantes en ZF-expression :

1. Définir la fonction (`sommeCarrePairs n`) qui calcule la somme des carrés des `n` premiers nombres pairs.

```
Prelude> sommeCarrePairs 1
4
Prelude> sommeCarrePairs 2
20
Prelude> sommeCarrePairs 8
816
```

2. Définir une fonction `moitiePairs` qui renvoie la moitié de chaque nombre pair de la liste. Vous pouvez utiliser les fonctions `div` et `mod` pour votre calcul.

```
Prelude>moitiePairs [0,2,1,7,8,56,17,18]
[0,1,4,28,9]
```

3. Définir une fonction `dansPlage` pour renvoyer tous les nombres dans une liste donnée appartenant à l'intervalle fermé donné par les deux premiers arguments.

```
Prelude>dansPlage 5 10 [1..15]
[5,6,7,8,9,10]
```

4. Définir une fonction `compterPositifs` pour compter les nombres positifs dans une liste (ceux strictement supérieurs à 0). Proposer deux définitions avec une expression en compréhension utilisant une fonction de la bibliothèque standard de Haskell puis une autre utilisant la récursivité `compterPositifsRec`.

```
Prelude>compterPositifs [0,1,-3,-2,8,-1,6]
3
```

Pourquoi il n'est pas possible de définir `compterPositifs` en utilisant uniquement la compréhension de liste, sans fonctions de bibliothèque ?

Exercice 4

En utilisant les ZF-expressions et en précisant la signature de chaque fonction :

- Définir une fonction `occurrencesPos` qui renvoie les positions de toutes les occurrences du caractère donné en deuxième argument dans la chaîne de caractères donnée en premier argument.

Exemple d'application :

```
*Main> occurrencePos "Programmation" 'a'
[5,8]
```

- Définir une fonction `prefixe` qui permet de vérifier si la chaîne de caractères donnée en premier argument contient comme préfixe la chaîne de caractères donnée en deuxième argument. Votre fonction doit retourner `faux` si la chaîne qui correspond au préfixe est plus longue que la chaîne à vérifier.

Exemple d'application :

```
*Main> prefix "14000" "14"
True
*Main> prefix "14000" "014"
False
```

- Définir une fonction `contient` qui prend deux chaînes de caractère en entrée et renvoie `True` si la première chaîne de caractères contient la seconde. Utiliser la fonction `prefixe` que vous avez défini précédemment.

```
*Main> "Programmation fonctionnelle" `contient` "fonc"
True
*Main> "Programmation fonctionnelle" `contient` "foncttio"
False
```
